

模型微调与数据集构建实战

一、为何需要微调？

模型微调与数据集构建实战

一、为何需要微调？

微调的核心价值

提示工程和 R A G 均未改变模型的底层参数与知识边界，而微调通过定向调整模型参数实现：

价值维度 说明

领域适配 让通用模型掌握垂直领域知识（如法律条文解读、工业设备故障诊断）

精度提升 针对细分任务，通过专项数据训练，降低模型 " 幻觉 " 与错误率

效率优化 微调后无需每次加载复杂提示或检索外部文档，响应速度更快

风格定制 让模型输出符合特定格式、语气或品牌风格

预训练 v s 微调

对比维度 预训练 微调

核心目标 学习通用知识（语言逻辑、常识） 适配特定任务（医疗文本分类、数学解题）

数据要求 大规模通用数据，无需标注 小规模任务相关数据，需标注

训练方式 自监督 / 无监督 有监督

参数更新 全量参数训练 部分 / 全量参数更新

资源消耗 极高（千亿级 T o k e n、数百张 G P U） 较低（万级 T o k e n、单张 / 几张 G P U）

微调的详细理论分析

迁移学习为什么有效？——特征转移假设

迁移学习的有效性基于一个核心假设：源领域学到的特征对目标领域是有用的。具体来说：

预训练模型学到的知识层次：

高层特征：任务特定		← 微调时主要修改
（情感极性、问答模式）		

中层特征：语义组合	← 微调时部分调整
(句法结构、逻辑关系)	
低层特征：通用语言特征	← 微调时通常保持
(词形、词性、基本语法)	

微调与特征提取的对比：

方面 特征提取 (Feature Extraction) 微调 (Fine-Tuning)

参数更新 仅训练新加的分类头 更新部分或全部参数

冻结策略 冻结预训练模型全部参数 可选择冻结 / 解冻不同层

效果 对相似任务效果好 对差异大的任务效果更好

过拟合风险 低 较高 (需要更多正则化)

训练时间 快 慢

适用场景 数据少、任务与预训练相似 数据较多、任务差异大

灾难性遗忘问题：

微调时如果学习率过大或训练太久，模型可能 " 忘记 " 预训练学到的通用知识，只在特定任务上表现好，泛化能力大幅下降。

灾难性遗忘的表现：

微调前：模型能流畅生成各种话题的内容

微调后 (过度)：模型只能生成特定领域的内容，其他任务表现急剧下降

解决方案：

- 1 . 使用较小的学习率 (比预训练小 $10 - 100$ 倍)
- 2 . 差异化学习率 (低层小、高层大)
- 3 . 渐进式解冻 (先训练高层，再逐步解冻低层)
- 4 . 使用 LoRA / Adapter 等参数高效方法 (天然防止灾难性遗忘)
- 5 . 正则化 (L2、Dropout、Early Stopping)
- 6 . 数据混合 (微调数据中混入部分通用数据)

- - -

二、全参微调 (Full Fine-Tuning) 详细实践

2.1 不同优化器的选择

```
from transformers import get_scheduler, AdamW
```

1. AdamW (最常用)

在标准Adam基础上解耦了权重衰减, 是微调LLM的默认选择

```
optimizer = AdamW(
    model.parameters(),
    lr=2e-5, # 微调常用学习率
    weight_decay=0.01, # 权重衰减 (防止过拟合)
    betas=(0.9, 0.999),
    eps=1e-8
)
```

2. SGD + Momentum (较少用于LLM, 但在某些CV任务中效果更好)

```
optimizer = torch.optim.SGD(
    model.parameters(),
    lr=1e-3,
    momentum=0.9,
    weight_decay=0.01
)
```

3. Adafactor (节省显存的优化器, 适合大模型)

```
from transformers import Adafactor
optimizer = Adafactor(
    model.parameters(),
    lr=2e-5,
    relative_step=False, # 需要手动设置lr
    scale_parameter=False,
    warmup_init=False
)
```

4. 8-bit Adam (进一步节省显存)

```
import bitsandbytes as bnb
optimizer = bnb.optim.Adam8bit(
model.parameters(),
lr=2e-5,
weightdecay=0.01
)
```

2.2 学习率设置策略

微调LLM的学习率经验值：

模型规模 推荐学习率 说明

< 1B	$5e-5 \sim 1e-4$	可以稍大
1B - 7B	$1e-5 \sim 5e-5$	常用范围
7B - 13B	$5e-6 \sim 2e-5$	偏小
> 13B	$1e-6 \sim 1e-5$	需要更小

差异化学习率：对不同层使用不同学习率

```
def getdifferentiallrgroups(model, baselr=2e-5,
" " " 低层使用更小的学习率，高层使用更大的学习率 " " "
paramgroups = []

# 假设模型有N层，越靠前的层学习率越小
for name, param in model.namedparameters():
    if not param.requires_grad:
        continue

# 根据层号确定学习率缩放
    if 'embed' in name or 'layer.0' in name or 'layer
paramgroups.append({'params': param, 'lr': baselr
    elif 'layer.2' in name or 'layer.3' in name:
paramgroups.append({'params': param, 'lr': baselr
    else:
paramgroups.append({'params': param, 'lr': baselr

return paramgroups
```

使用

```
optimizer = AdamW(getdifferentiallrgroups(model,
```

2.3 渐进式解冻 (Gradual Unfreezing)

渐进式解冻：从高层开始训练，逐步解冻更多层

```
def gradualunfreeze(model, currentepoch, unfreeze
"""
```

Args:

```
unfreezeschedule: {epoch: [要解冻的层名关键词]}
"""
```

```
if currentepoch in unfreezeschedule:
    keywords = unfreezeschedule[currentepoch]
    for name, param in model.namedparameters():
        for kw in keywords:
            if kw in name:
                param.requiresgrad = True
            print(f" 解冻: {name}")
```

示例：3阶段解冻

```
schedule = {
    0: ['classifier', 'lmhead'], # 第0轮：只训练分类头
    2: ['layer.3', 'layer.4'], # 第2轮：解冻最后2层
    4: ['layer.1', 'layer.2'], # 第4轮：解冻更多层
}

for epoch in range(numepochs):
    gradualunfreeze(model, epoch, schedule)
    trainoneepoch(model, ...)

- - -
```

三、LoRA：低秩适应深入

3.1 LoRA的详细数学推导

L o R A 的核心假设：模型微调时的参数更新量 ΔW 是低秩的。

原始权重矩阵： $W \in \mathbb{R}^{(d \times k)}$ （冻结不变）

更新量： $\Delta W = BA$ ，其中 $B \in \mathbb{R}^{(d \times r)}$ ， $A \in \mathbb{R}^{(r \times k)}$ ， $r \ll \min(d, k)$

前向传播： $y = (W + \Delta W)x = Wx + BAx = Wx + B(Ax)$

为什么低秩分解有效？

1. 微调只需调整模型在特定子空间的行为
2. 预训练模型的权重已经足够好，更新量是 "小而精" 的
3. 类比：图像压缩（SVD分解），少量主成分就能保留主要信息

数学直觉：

- 原始参数量： $d \times k$
- L o R A 参数量： $d \times r + r \times k = r(d + k)$
- 当 $r \ll \min(d, k)$ 时，参数量大幅减少
- 例如： $d = k = 4096$ ， $r = 16$

原始： $4096 \times 4096 = 16,777,216$

L o R A： $16 \times (4096 + 4096) = 131,072$ （仅 0.78%）

缩放因子的作用：

实际输出： $y = Wx + (\alpha / r) \times BAx$

其中 α 是 L o R A 的缩放因子（l o r a a l p h a）：

- α / r 控制 L o R A 更新相对于原始权重的比例
- $\alpha = r$ 时，缩放因子为 1（默认）
- $\alpha = 2r$ 时，L o R A 的影响力加倍
- 较大的 $\alpha / r \rightarrow$ 更大的更新幅度，但可能不稳定
- 较小的 $\alpha / r \rightarrow$ 更保守的更新，更稳定

选择建议：

- $r = 8$ 时， $\alpha = 16$ （即 $\alpha / r = 2$ ）是常见选择
- $r = 16$ 时， $\alpha = 32$ （即 $\alpha / r = 2$ ）也是常见选择
- 保持 $\alpha / r \approx 2$ 是一个不错的起点

3.2 LoRA的秩选择实验

LoRA秩选择经验法则：

场景 推荐秩 原因

简单任务（单标签分类） $r = 4 \sim 8$ 任务简单，低秩足够

中等任务（多标签分类、简单SFT） $r = 8 \sim 16$ 平衡效果和效率

复杂任务（代码生成、数学推理） $r = 16 \sim 64$ 需要更大的更新空间

多任务微调 $r = 32 \sim 128$ 需要覆盖多个子空间

实验方法：从 $r = 8$ 开始，逐步增大，观察验证集性能

```
from peft import LoraConfig, get_peft_model

for r in [4, 8, 16, 32, 64]:
    config = LoraConfig(
        r=r,
        lora_alpha=r*2,
        target_modules=["qproj", "vproj", "kproj", "oproj"],
        loradropout=0.05,
        task_type="CAUSALLM"
    )
    model = get_peft_model(base_model, config)
    # 训练并记录验证集性能...
    print(f"r={r}: trainable_params={model.print_trainable_params}")
```

3.3 目标模块选择策略

不同目标模块的选择策略：

策略1：仅注意力（最常用，效果好）

```
target_modules = ["qproj", "vproj"] # 最小配置
target_modules = ["qproj", "kproj", "vproj", "oproj"]
```

策略2：注意力 + FFN（效果更好，参数更多）

```
target_modules = ["qproj", "kproj", "vproj", "oproj",
                  "gateproj", "upproj", "downproj"]
```

策略3：仅FFN（某些场景有效）

```
targetmodules = ["gateproj", "upproj", "downproj"]
```

选择建议：

资源 推荐策略

显存有限 $qproj + vproj$

平衡 $q+k+v+oproj$

追求最佳效果 注意力全 + FFN

查看模型的所有线性层名称

```
for name, module in model.namedmodules():
    if isinstance(module, torch.nn.Linear):
        print(name, module.infeatures, module.outfeatures)
```

3.4 LoRA vs 全参微调的效果对比

典型对比结果（以7B模型为例）：

方法 可训练参数 显存需求 训练速度 效果（相对全参）

全参微调 7B （100%） ~120GB 慢 100% （基准）

LoRA $r=8$ ~4M （0.06%） ~20GB 快 ~95-98%

LoRA $r=16$ ~8M （0.11%） ~22GB 快 ~97-99%

LoRA $r=64$ ~32M （0.46%） ~28GB 中 ~99-100%

LoRA $r=128$ ~64M （0.91%） ~35GB 中 ~100%+

关键发现：

1. $r=16 \sim 64$ 的LoRA在大多数任务上已接近全参微调
2. 某些任务上LoRA甚至优于全参微调（正则化效应）
3. LoRA训练速度快2-5倍，显存减少70-80%

3.5 LoRA合并（Merge）的原理和实现

LoRA训练完成后，可以将LoRA权重合并回原始模型

合并后推理无额外开销


```
from peft import PeftModel
```

方法1：使用PEFT的合并功能

```
basemodel = AutoModelForCausalLM.from_pretrained('...')
model = PeftModel.from_pretrained(basemodel, "lora_path")
mergedmodel = model.merge_and_unload() # 合并后LoRA权重融入基模型
mergedmodel.save_pretrained("mergedmodel")
```

方法2：手动合并

合并公式： $W_{new} = W + (\alpha / r) B @ A$

```
def manual_merge_lora(model, lorapath):
    statedict = torch.load(lorapath)
    for name, param in model.named_parameters():
        loraAkey = f"{name}.loraA.weight"
        loraBkey = f"{name}.loraB.weight"
        if loraAkey in statedict and loraBkey in statedict:
            A = statedict[loraAkey] # (r, in_features)
            B = statedict[loraBkey] # (out_features, r)
            scale = alpha / r # 缩放因子
            param.data += scale * B @ A # 合并
```

多LoRA融合（多个任务的LoRA权重融合）

方法1：线性插值

```
mergedweight = w1 * lora1weight + w2 * lora2weight # 权重插值
```

方法2：TIES merging（更高级的融合方法）

1. 修剪：只保留每个LoRA中最大的k%参数
2. 符号选择：按多数投票决定每个参数的符号
3. 合并：对齐符号后取平均

- - -

四、QLoRA：量化LoRA详细讲解

4.1 4-bit量化的原理（NF4数据类型）

Q L o R A 的核心创新是 N F 4 (N o r m a l F l o a t 4) 数据类型，专为正态分布的权重设计。

传统 4 - b i t 量化（均匀量化）：

- 将 F P 3 2 的值域均匀划分为 1 6 个区间
- 问题：权重通常服从正态分布，均匀量化浪费精度

N F 4 量化（非均匀量化）：

- 基于正态分布的分位数定义 1 6 个量化级别
- 每个区间包含相同概率的值
- 优势：在权重密集区域分配更多量化级别

N F 4 的量化级别（来自 $N(0, 1)$ 的分位数）：

[- 1 . 0 , - 0 . 6 9 6 , - 0 . 4 7 6 , - 0 . 2 9 7 , - 0 . 1 4 7 , 0 , 0 . 1 4 7 ,
0 . 4 7 6 , 0 . 6 9 6 , 1 . 0 , . . .]

量化过程：

- 1 . 确定权重的绝对值最大值
- 2 . 将权重归一化到 [- 1 , 1]
- 3 . 用 N F 4 分位数近似每个权重值
- 4 . 存储量化索引 (4 b i t) + 缩放因子

反量化：量化索引 $\times$ 缩放因子 $\rightarrow$ F P 1 6 / B F 1 6 用于计算

4 . 2 双重量化

Q L o R A 的另一个创新：双重量化

量化后的缩放因子本身也可以被量化

普通量化：权重 $\rightarrow$ 4 b i t + 缩放因子 (F P 3 2)

双重量化：权重 $\rightarrow$ 4 b i t + 缩放因子 (F P 3 2) $\rightarrow$ 缩放因子量化为 F P 3 2 + 二级缩放因子

效果：每个参数额外节省约 0 . 3 7 b i t 的显存

Q L o R A 的完整配置

```
from transformers import BitsAndBytesConfig

bnbconfig = BitsAndBytesConfig(
```

```
load_in4bit=True, # 4-bit 量化加载
bnb4bitquanttype="nf4", # NF4 数据类型
bnb4bitcomputedtype=torch.bfloat16, # 计算时使用 BF16
bnb4bitusedoublequant=True, # 启用双重量化
)
```

加载模型

```
model = AutoModelForCausalLM.from_pretrained(
    "Qwen/Qwen2.5-7B",
    quantizationconfig=bnbconfig,
    device_map="auto"
)
```

配置 LoRA

```
from peft import LoraConfig, get_peft_model

loracconfig = LoraConfig(
    r=16,
    lora_alpha=32,
    target_modules=["qproj", "vproj", "kproj", "oproj"],
    loradropout=0.05,
    tasktype="CAUSALLM"
)
model = get_peft_model(model, loracconfig)
model.print_trainable_parameters()
```

输出: trainable params: 13,107,200 all params: 3,540,000

4.3 QLoRA vs LoRA 的显存对比

以 7 B 模型为例的显存对比：

配置	参数存储	梯度	优化器状态	总显存
全参微调 (FP32)	28 GB	28 GB	56 GB	~120 GB
全参微调 (BF16)	14 GB	14 GB	28 GB	~60 GB
LoRA (BF16)	14 GB	+ 0.05 GB	0.05 GB	0.1 GB ~20 GB

Q L o R A (N F 4) 3 . 5 G B + 0 . 0 5 G B 0 . 0 5 G B 0 . 1 G B ~ 8 G B

关键差异：

- Q L o R A 比 L o R A 节省约 6 0 - 7 0 % 显存
- Q L o R A 使得 7 B 模型可以在 R T X 4 0 9 0 (2 4 G B) 上微调
- Q L o R A 使得 1 3 B 模型可以在 A 1 0 0 (4 0 G B) 上微调
- Q L o R A 训练速度比 L o R A 慢约 1 0 - 2 0 % (量化和反量化的开销)
- - -

五、Adapter Tuning 详细讲解

5.1 不同的Adapter 设计

```
import torch
import torch.nn as nn
```

===== Houlsby Adapter =====

在每个Transformer层的注意力和FFN之后都插入Adapter

```
class HoulsbyAdapter(nn.Module):
```

```
def __init__(self, hidden_size, adapter_size=64):
```

```
    super().__init__()
```

```
    self.downproj = nn.Linear(hidden_size, adapter_size)
```

```
    self.upproj = nn.Linear(adapter_size, hidden_size)
```

```
    self.activation = nn.GELU()
```

```
    self.init_weights()
```

```
def init_weights(self):
```

```
    # 初始化：下投影用小随机值，上投影用零
```

```
    # 这样初始时Adapter输出为0，不影响原始模型
```

```
    nn.init.normal_(self.downproj.weight, std=1e-3)
```

```
    nn.init.zeros_(self.upproj.weight)
```

```
    nn.init.zeros_(self.upproj.bias)
```

```
def forward(self, x):
```

```
    residual = x
```

```

x = self.downproj(x)
x = self.activation(x)
x = self.upproj(x)
return residual + x # 残差连接

```

===== Pfeiffer Adapter =====

只在FFN之后插入Adapter（比Houlsby少一半）

结构相同，但插入位置不同

===== Adapter插入方式 =====

```

class TransformerLayerWithAdapter(nn.Module):
def init(self, originallayer, adaptersize=64):
super().init()
self.originallayer = originallayer
hiddensize = originallayer.hiddensize

```

Houlsby: 两个Adapter

```

self.adapterafterattn = HoulsbyAdapter(hiddensize)
self.adapterafterffn = HoulsbyAdapter(hiddensize)

```

```

def forward(self, hiddenstates, ...):

```

原始注意力

```

attnoutput = self.originallayer.attention(hiddensize,
hiddenstates = hiddenstates + attnoutput

```

Adapter

```

hiddenstates = self.adapterafterattn(hiddenstates)

```

原始FFN

```

ffnoutput = self.originallayer.ffn(hiddenstates)
hiddenstates = hiddenstates + ffnoutput

```

Adapter

```

hiddenstates = self.adapterafterffn(hiddenstates)

```

```

return hiddenstates

```

5.2 Adapter的推理开销

Adapter的推理影响：

Adapter类型 额外参数 推理延迟增加 说明

Houlsby $2 \times d \times r$ per layer $\sim 5 - 10\%$ 每层2个Adapter

Pfeiffer $1 \times d \times r$ per layer $\sim 2 - 5\%$ 每层1个Adapter

LoRA $2 \times d \times r$ per target $\sim 0\%$ (合并后) 合并后无额外开销

IA3 $3 \times d$ per layer $\sim 0\%$ 仅向量缩放

关键对比：

- LoRA合并后零推理开销 → 部署首选
- Adapter始终有额外推理开销 → 不适合对延迟敏感的场景
- Adapter的优势：可以动态切换不同任务的Adapter（多任务服务）

- - -

六、Prompt Tuning和P-Tuning详细讲解

===== Prompt Tuning =====

在输入前添加可学习的"虚拟token"，只训练这些token的嵌入

```
from peft import PromptTuningConfig, get_peft_model

config = PromptTuningConfig(
    task_type="CAUSALLM",
    num_virtual_tokens=20, # 虚拟token数量
    token_dim=model.config.hidden_size,
    num_transformer_submodules=1,
    num_attention_heads=model.config.num_attention_heads,
    num_layers=model.config.num_hidden_layers,
)
```

```
model = get_peft_model(model, config)
```

可训练参数极少： $20 \times \text{hidden_size} \approx 20 \times 4096 = 81,920$

Prompt Tuning的工作原理：

原始输入：`[ token1, token2, ..., tokenN ]`

加上 Prompt：`[ virtual1, virtual2, ..., virtual20, token1, token2, ..., tokenN ]`

只训练 `virtual1 ~ virtual20` 的嵌入向量

虚拟 `token` 不对应词表中的任何词，是完全可学习的

===== P - Tuning v2 =====

在每一层都添加可学习的 `prompt`（深层 `prompt`），效果比 `Prompt Tuning` 好

```
from peft import PrefixTuningConfig
```

```
config = PrefixTuningConfig(  
    tasktype="CAUSALLM",  
    numvirtualtokens=20,  
    prefixprojection=True, # 使用投影层  
    projectiondim=512, # 投影维度  
)
```

P - Tuning v2 vs Prompt Tuning:

方面 Prompt Tuning P - Tuning v2

可学习参数位置 仅输入层 每一层

效果 小模型效果差 全面优于 Prompt Tuning

参数量 更少 更多

适合模型规模 > 10B 所有规模

经验：

Prompt Tuning 只适合大模型 (> 10B)，小模型效果很差

P - Tuning v2 在各种规模模型上都有效

对于 LLM 微调，LoRA 仍是首选

- - -

七、IA3 详细讲解

IA3: Injected Attention and Adapted Activations

通过学习向量对 Key、Value 和 FFN 输出进行缩放

核心公式：

注意力中： $K' = l_k \odot K$, $V' = l_v \odot V$ ($\odot$ 逐元素乘)

FFN中： $FFN' = l_{ff} \odot FFN_{output}$

其中 l_k , l_v , l_{ff} 是可学习的向量（不是矩阵！）

参数量对比（以 $d = 4096$ 为例）：

LoRA $r=16$: $2 \times 4096 \times 16 = 131,072$ per target module

IA3: 4096 per vector $\times 3 = 12,288$ per layer

IA3的参数量约为LoRA的 $1/10$

```
from peft import IA3Config, get_peft_model

config = IA3Config(
    task_type="CAUSALLM",
    target_modules=["kproj", "vproj", "downproj"], # 对注意力模块
    feedforward_modules=["downproj"], # 指定哪些是FFN模块
)

model = get_peft_model(model, config)
model.print_trainable_parameters()
```

IA3 优势：

1. 参数极少（LoRA的 $1/10$ ）
2. 推理时可以合并（向量缩放融入权重）
3. 适合极端资源受限场景

IA3 劣势：

1. 表达能力不如LoRA
2. 在复杂任务上效果可能不如LoRA

- - -

八、高效微调技术对比总结

技术名称	核心原理	可训练参数量	显存节省	推理开销	适用场景
------	------	--------	------	------	------

全参微调	更新所有参数	100%	无	无	数据充足、资源充足
------	--------	------	---	---	-----------

LoRA	低秩矩阵分解	0.1 - 1%	70 - 80%	无（合并后）	通用首选
QLoRA	4-bit量化+LoRA	0.1 - 1%	85 - 90%	无（合并后）	消费级GPU
Adapter	插入小网络	1 - 5%	60 - 70%	有	多任务切换
Prompt Tuning	可学习虚拟token	< 0.01%	90%+	极小	大模型简单任务
P-Tuning v2	深层可学习prompt	0.1 - 1%	85%	极小	中小模型
IA3	向量缩放	< 0.01%	90%+	无	极端资源受限
Freeze	冻结大部分层	10 - 30%	30 - 50%	无	简单任务

- - -

九、数据集构建完整流程

9.1 数据收集策略

===== 数据来源分类 =====

1. 内部业务数据（最佳来源）

- 客服对话记录
- 用户反馈和评价
- 内部知识库文档
- 专家标注数据

2. 公开数据集

- HuggingFace Datasets
- Kaggle
- PapersWithCode

3. 合成数据（LLM生成）

- 使用GPT-4 / Claude生成训练数据
- 自我指导（Self-Instruct）方法
- Evol-Instruct：进化式指令生成

===== 合成数据生成示例 =====

```
import json
from openai import OpenAI

client = OpenAI(apikey="your-key")
```

```
def generatetrainingdata(taskdescription, numsamples):
    """使用LLM生成训练数据"""
    data = []
    for i in range(numsamples):
        response = client.chat.completions.create(
            model="gpt-4",
            messages=[
                {"role": "system", "content": f"你是一个数据标注专家。{taskdescription}"},
                {"role": "user", "content": "请生成一条训练数据，包含输入和期望输出。"}
            ]
        )
        data.append({"text": response.choices[0].message.content})
    return data
```

批量生成并质量过滤

```
def generatewithqualitycheck(taskdesc, numsamples):
    rawdata = generatetrainingdata(taskdesc, numsamples)
    # 质量过滤：使用另一个LLM或规则过滤低质量数据
    filtered = [d for d in rawdata if checkquality(d)]
    return filtered[:numsamples]
```

9.2 数据标注工具

常用数据标注工具对比：

工具 特点 适用任务 开源

Label Studio 全功能、支持多种数据类型 图像、文本、音频、视频

Doccano 轻量、专注NLP 文本分类、序列标注、翻译

Argilla LLM数据标注专用 SFT、RLHF、偏好排序

Prodigy 主动学习、高效 NLP、CV

Roboflow CV专用 目标检测、分割 部分

Label Studio快速上手：

1. `pip install label-studio`

2. label-studio start
3. 浏览器打开 `http://localhost:8080`
4. 创建项目 → 选择标注模板 → 导入数据 → 开始标注
5. 导出标注结果 (JSON / CSV / COCO等格式)

9.3 数据清洗和质量控制

```
import re
import json
from collections import Counter

def cleanqualitycontrol(datalist):
    """ 数据清洗和质量控制Pipeline """
    cleaned = []
    stats = {"total": len(datalist), "removed": 0, "reasons": {}}

    for item in datalist:
        # 1. 去除空数据
        if not item.get('instruction') or not item.get('output'):
            stats["removed"] += 1
            stats["reasons"]["empty"] += 1
            continue

        # 2. 去除过短 / 过长的数据
        outputlen = len(item['output'])
        if outputlen < 10:
            stats["removed"] += 1
            stats["reasons"]["tooshort"] += 1
            continue
        if outputlen > 4096:
            stats["removed"] += 1
            stats["reasons"]["toolong"] += 1
            continue

        # 3. 去除重复数据
```

(在外部使用 s e t 或 m i n h a s h 去重)

4 . 去除低质量数据（包含大量特殊字符等）

```
specialcharratio = len(re.findall(r'[\^\\w\\s\\u4e00-
item['output']))) / max(outputlen, 1)
if specialcharratio > 0.3:
stats["removed"] += 1
stats["reasons"]["specialchars"] += 1
continue
```

5 . 去除模板化回答

```
templatephrases = [ "我无法回答", "作为AI", "I cannot", '
if any(phrase in item['output'] for phrase in ten
stats["removed"] += 1
stats["reasons"]["template"] += 1
continue
```

```
cleaned.append(item)
```

```
print(f"清洗结果: {stats['total']} → {len(cleaned)}")
print(f"移除原因: {dict(stats['reasons'])}")
return cleaned
```

去重 (M i n H a s h L S H)

```
def deduplicate_minhash(texts, numperm=128, thresh=0.9):
    """使用MinHash进行近似去重"""
```

```
from datasketch import MinHash, MinHashLSH
```

```
lsh = MinHashLSH(threshold=threshold, numperm=numperm)
uniquetexts = []
```

```
for i, text in enumerate(texts):
    mh = MinHash(numperm=numperm)
    for word in text.split():
        mh.update(word.encode('utf-8'))
```

```

if not lsh.query(mh): # 没有相似的
    lsh.insert(str(i), mh)
    uniquetexts.append(text)

return uniquetexts

```

9.4 数据格式设计

===== Alpaca 格式（最常用的 SFT 格式）=====

```

alpacaformat = {
    "instruction": "将以下句子翻译成英文",
    "input": "今天天气很好",
    "output": "The weather is very nice today."
}

```

无 input 的情况

```

alpacanoinput = {
    "instruction": "写一首关于春天的诗",
    "input": "",
    "output": "春风拂面花香来..."
}

```

===== ShareGPT 格式（对话格式）=====

```

sharegptformat = {
    "conversations": [
        {"from": "human", "value": "请解释什么是机器学习"},
        {"from": "gpt", "value": "机器学习是人工智能的一个分支..."},
        {"from": "human", "value": "它和深度学习有什么区别?"},
        {"from": "gpt", "value": "深度学习是机器学习的子集..."}
    ]
}

```

===== 转换为训练格式 =====

```

def formattoprompt(sample, formattype="alpaca"):
    """将数据转换为模型训练时的prompt格式"""

```

```

if formattype == "alpaca":
    if sample.get("input"):
        prompt = f"### Instruction:\n{sample['instruction']}\n\n"
        prompt += f"### Input:\n{sample['input']}\n\n"
    else:
        prompt = f"### Instruction:\n{sample['instruction']}\n\n"
        prompt += f"### Response:\n{sample['output']}\n\n"
    return prompt

elif formattype == "sharegpt":
    prompt = ""
    for conv in sample["conversations"]:
        if conv["from"] == "human":
            prompt += f"< imstart >user\n{conv['value']}< imend >\n"
        else:
            prompt += f"< imstart >assistant\n{conv['value']}\n"
    return prompt

```

选择建议：

格式 适用场景 优势

Alpaca 单轮指令微调 简单、通用

ShareGPT 多轮对话微调 支持多轮对话

9.5 数据增强技术

===== 文本数据增强 =====

1. 回译增强 (Back Translation)

```

def backtranslation(text, sourcelang='zh', targetlang='en'):
    """ 中文→英文→中文，获得释义相同但表达不同的数据 """
    translated = translate(text, sourcelang, targetlang)
    backtranslated = translate(translated, targetlang, sourcelang)
    return backtranslated

```

2. 同义词替换

```

import synonyms # pip install synonyms

def synonymreplacement(text, replaceratio=0.3):
    words = jieba.lcut(text)
    nreplace = max(1, int(len(words) * replaceratio))
    indices = random.sample(range(len(words)), min(nreplace, len(words)))

    for idx in indices:
        syns = synonyms.nearby(words[idx])
        if syns[0]:
            words[idx] = random.choice(syns[0])
    return ' '.join(words)

```

3. Self-Instruct: 用LLM生成多样化数据

```

def selfinstructevolve(seedtasks, numiterations=3):
    """Evol-Instruct: 逐步增加指令难度"""
    currenttasks = seedtasks
    for i in range(numiterations):
        evolved = []
        for task in currenttasks:
            # 让LLM增加约束、加深复杂度
            prompt = f"请将以下指令变得更加复杂和具体：\n{task}"
            newtask = llmgenerate(prompt)
            evolved.append(newtask)
        currenttasks.extend(evolved)
    return currenttasks

```

9.6 数据质量评估

```

def assessdataquality(dataset):
    """评估数据集质量"""
    metrics = {}

    # 1. 多样性评估
    # 使用n-gram重复率衡量

```

```

from collections import Counter
allngrams = []
for sample in dataset:
    tokens = sample['output'].split()
    bigrams = list(zip(tokens, tokens[1:]))
    allngrams.extend(bigrams)

ngramcounts = Counter(allngrams)
duplicateratio = sum(c for c in ngramcounts.values())
metrics['bigramduplicateratio'] = duplicateratio

# 2. 长度分布
lengths = [len(s['output']) for s in dataset]
metrics['avglength'] = sum(lengths) / len(lengths)
metrics['lengthstd'] = (sum((l - metrics['avglength'])**2 for l in lengths))**0.5

# 3. 指令多样性
instructions = [s['instruction'] for s in dataset]
uniqueinstructions = len(set(instructions))
metrics['instructiondiversity'] = uniqueinstructions

# 4. 质量评分（使用LLM评分）
# 随机抽样100条，用GPT-4评分
sampleforreview = random.sample(dataset, min(100, len(dataset)))
# scores = [llmqualityscore(s) for s in sampleforreview]
# metrics['avgqualityscore'] = sum(scores) / len(scores)

return metrics

- - -

```

十、微调不同任务的详细教程

10.1 文本分类微调

```

from transformers import AutoModelForSequenceClassification

```



```
TrainingArguments, Trainer
from peft import LoraConfig, get_peft_model
```

加载模型（添加分类头）

```
model = AutoModelForSequenceClassification.from_pretrained(
    "Qwen/Qwen2.5-1.8B",
    num_labels=2, # 二分类
)
```

配置LoRA

```
lora_config = LoraConfig(
    r=8,
    lora_alpha=16,
    target_modules=["qproj", "vproj"],
    task_type="SEQCLS", # 序列分类任务
)
model = get_peft_model(model, lora_config)
```

数据预处理

```
tokenizer = AutoTokenizer.from_pretrained("Qwen/Qwen2.5-1.8B")
def preprocess(examples):
    return tokenizer(examples["text"], truncation=True, padding="max_length")

tokenized_dataset = dataset.map(preprocess, batched=True)
```

训练

```
training_args = TrainingArguments(
    output_dir="./clsmodel",
    per_device_train_batch_size=16,
    num_train_epochs=3,
    learning_rate=2e-5,
    evaluation_strategy="epoch",
    save_strategy="epoch",
    load_best_model_at_end=True,
)
```

```

trainer = Trainer(model=model, args=trainingargs,
traindataset=tokenizeddataset["train"],
evaldataset=tokenizeddataset["test"])
trainer.train()

```

10.2 对话微调 (SFT)

```

from transformers import AutoModelForCausalLM, AutoTokenizer
from peft import LoraConfig, get_peft_model, TaskType

```

加载模型

```

model = AutoModelForCausalLM.from_pretrained(
    "Qwen/Qwen2.5-7B",
    torch_dtype=torch.bfloat16,
    device_map="auto"
)

```

LoRA 配置 (对话微调建议更大的 r)

```

lora_config = LoraConfig(
    r=16,
    lora_alpha=32,
    target_modules=["qproj", "kproj", "vproj", "opproj",
"gateproj", "upproj", "downproj"],
    loradropout=0.05,
    task_type=TaskType.CAUSALLM,
)
model = get_peft_model(model, lora_config)

```

对话数据格式化

```

def format_chat(example):
    """ 格式化为 ChatML 格式 """
    messages = []
    for conv in example["conversations"]:
        if conv["from"] == "human":
            messages.append({"role": "user", "content": conv["value"]})

```

```

else:
    messages.append({"role": "assistant", "content": 
    return tokenizer.apply_chat_template(messages, token

```

SFT训练参数

```

from trl import SFTTrainer, SFTConfig

sftconfig = SFTConfig(
    output_dir="/sftmodel",
    per_device_train_batch_size=4,
    gradient_accumulation_steps=8, # 有效batchsize=32
    num_train_epochs=3,
    learning_rate=2e-5,
    max_seq_length=2048,
    logging_steps=10,
    save_strategy="epoch",
    bf16=True,
)
trainer = SFTTrainer(
    model=model,
    args=sftconfig,
    train_dataset=formatteddataset,
    formatting_func=format_chat,
)
trainer.train()

```

10.3 代码生成微调

代码微调的关键：数据质量和格式

推荐使用Code Alpaca格式

```

codedataformat = {
    "instruction": "写一个Python函数，实现二分查找",
    "input": "",
    "output": "def binarysearch(arr, target):\n    left,

```

```

right:\n mid = (left + right) // 2\n if arr[mid]
target:\n left = mid + 1\n else:\n right = mid -
}

```

代码微调建议：

1. 使用代码专用 tokenizer (如 CodeLlama 的 tokenizer)
2. 较大的 max seq length (代码通常较长, 建议 2048 - 4096)
3. 较大的 LoRA rank ($r = 32 \sim 64$)
4. 学习率稍低 ($1e-5 \sim 2e-5$)
5. 数据中包含多种编程语言和难度级别

10.4 多模态微调

以 LLaVA 为例的多模态微调

```

from transformers import LlavaForConditionalGeneration

model = LlavaForConditionalGeneration.from_pretrained(
    "llava-hf/llava-1.5-7b-hf",
    torch_dtype=torch.bfloat16,
    device_map="auto"
)

```

多模态 LoRA 配置

```

from peft import LoraConfig

lora_config = LoraConfig(
    r=16,
    lora_alpha=32,
    target_modules=["qproj", "vproj"], # 通常只对语言模型部分做 LoRA
    task_type="CAUSALLM",
)

```

只对 language model 部分应用 LoRA

```

model.language_model = get_peft_model(model.language_model, lora_config)

- - -

```

十一、评估微调效果的方法

===== 自动评估 =====

1. 困惑度 (Perplexity) — 评估语言模型质量

```
def computeperplexity(model, tokenizer, texts, device):
    import math
    model.eval()
    totalloss = 0
    totaltokens = 0
    for text in texts:
        inputs = tokenizer(text, return_tensors='pt').to(device)
        with torch.no_grad():
            outputs = model(*inputs, labels=inputs['input_ids'])
            totalloss += outputs.loss.item()
            totaltokens += inputs['input_ids'].size(1)
    return math.exp(totalloss / totaltokens)
```

2. 任务特定指标

分类: accuracy, F1, precision, recall

生成: BLEU, ROUGE, BERTScore

代码: pass@k (HumanEval)

3. 基准测试

通用能力: MMLU, C-Eval

数学: GSM8K, MATH

代码: HumanEval, MBPP

===== 人工评估 =====

1. 盲测对比 (A/B Test): 微调前 vs 微调后

2. 评分法: 1 - 5分评估回答质量

3. 偏好排序: 多个模型输出排序

===== 常见评估陷阱 =====

1. 测试集泄漏: 确保训练集和测试集不重叠

2. 指标欺骗: 高 BLEU 不等于高质量

3 . 领域偏移：评估数据要与实际应用场景匹配

4 . 只看任务指标，忽略通用能力退化

- - -

十二、LLaMA Factory详细使用教程

===== 安装 =====

```
git clone https://github.com/hiyouga/LLaMA-Factory
cd LLaMA-Factory
pip install -e ".[torch,metrics]"
```

===== WebUI启动 =====

```
llamafactory-cli webui
```

浏览器打开 <http://localhost:7860>

===== 命令行训练 =====

创建配置文件 `loratrain.yaml`

```
loratrain.yaml
```

```
modelnameorpath: Qwen/Qwen2.5-7B
```

```
stage: sft
```

```
dotrain: true
```

```
finetuningtype: lora
```

```
loratarget: qproj,vproj,kproj,opproj
```

```
lorarank: 16
```

```
loraalpha: 32
```

```
loradropout: 0.05
```

```
dataset: alpacaendemo # 数据集名称（在datasetinfo.json中定义）
```

```
template: qwen
```

```
cutoffflen: 2048
```

```
outputdir: saves/qwen-lora-sft
```

```
perdevicetrainbatchsize: 4
```

```
gradientaccumulationsteps: 8
```

```
lrschedulertype: cosine
```

```
loggingsteps: 10
savesteps: 500
numtrainepochs: 3
learningrate: 2.0e-5
bf16: true
plotloss: true
```

启动训练

```
llamafactory-cli train loratrain.yaml
```

==== LoRA合并 ====

```
merge.yaml
```

```
llamafactory-cli merge merge.yaml
```

==== 推理测试 ====

```
llamafactory-cli chat inference.yaml
```

自定义数据集注册：

```
// datasetinfo.json
{
  "mydataset": {
    "filename": "mydata.json",
    "formatting": "alpaca",
    "columns": {
      "prompt": "instruction",
      "query": "input",
      "response": "output"
    }
  }
}

---
```

十三、Unsloth加速微调教程

===== 安装 =====

```
pip install unsloth
```

===== 使用Unsloth进行LoRA微调 =====

```
from unsloth import FastLanguageModel
```

加载模型 (Unsloth优化版, 自动4-bit量化)

```
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name="unsloth/Qwen2.5-7B",
    max_seq_length=2048,
    dtype=None, # 自动检测
    load_in_4bit=True, # 4-bit量化
)
```

添加LoRA适配器

```
model = FastLanguageModel.get_peft_model(
    model,
    r=16,
    lora_alpha=32,
    lora_dropout=0.05,
    target_modules=["qproj", "kproj", "vproj", "oproj",
                    "gateproj", "upproj", "downproj"],
)
```

训练 (与标准HuggingFace Trainer相同)

```
from trl import SFTTrainer
from transformers import TrainingArguments

trainer = SFTTrainer(
    model=model,
    train_dataset=dataset,
    args=TrainingArguments(
        per_device_train_batch_size=4,
        gradient_accumulation_steps=8,
```



```
numtrainepochs = 3 ,  
learningrate = 2e - 5 ,  
outputdir = " outputs " ,  
bf16 = True ,  
) ,  
)  
trainer . train ( )
```

Unsloth优势：

- 1 . 训练速度提升 2 - 5 倍
- 2 . 显存减少约 6 0 %
- 3 . 无需手动配置量化
- 4 . 兼容 Hugging Face 生态系统

- - -

十四、模型训练状态判断

模型状态 损失表现 解决策略

训练失败 训练损失不变甚至上升 ①检查数据格式 ②调整学习率 ③检查模型配置

欠拟合 两者均下降但仍较高 ①增加训练轮次 ②扩大数据集 ③增大 LoRA 的秩

过拟合 训练损失下降，验证损失上升 ①减少训练轮次 ②增加数据多样性 ③增大 dropout 率 ④调

灾难性遗忘 微调任务好，其他任务差 ①降低学习率 ②混入通用数据 ③使用 LoRA 而非全参

- - -

十五、常见微调问题的排查

问题排查清单：

- 1 . Loss 不下降

→ 检查数据格式是否正确 (instruction / output 是否对齐)

→ 检查 tokenizer 的 max length 是否足够

→ 检查学习率是否太小 (试试 2e - 5)

→ 检查是否所有参数都被冻结了

2. `Loss`为NaN

- 降低学习率
- 检查数据是否包含异常值
- 使用bf16而非fp16
- 检查label是否正确对齐

3. 生成重复内容

- 增加数据多样性
- 减小学习率
- 增加loradropout
- 减少训练轮次

4. 输出格式不对

- 在instruction中明确指定输出格式
- 增加格式正确的训练样本
- 使用system prompt约束格式

5. 显存不足(OOM)

- 使用QLoRA替代LoRA
- 减小batchsize + 增大gradientaccumulationsteps
- 减小maxseq_length
- 使用Unsloth加速

6. 中文效果差

- 确保使用中文预训练模型
- 增加中文训练数据比例
- 检查tokenizer是否支持中文

- - -

核心总结：微调的核心步骤是 " 选模型→选方法（LoRA）→准备数据→训练→评估→部署 "。LoRA是资源受限下的首选，QLoRA进一步降低显存需求。数据质量比数量更重要，至少需要1000+条优质数据。选择微调方案需考虑资源、任务复杂度和推理需求：LoRA适合通用场景，QLoRA适合消费级GPU，Adapter适合数据充足的场景。